Regularization and Validation

Contents

5.1 Regularization **1**
5.1.1 L2 regularization: weight decay . . . 2
5.1.1.1 Regularized linear neuron for regression: analytical solution . . . 2
5.1.2 L1 regularization . . . 3

5.2 Validation **4**
5.2.1 Overfitting vs. Underfitting . . . 5
5.2.1.1 Dealing with overfitting/underfitting . . . 6
5.2.1.2 Different views of the bias-variance tradeoff . . . 6
5.2.2 Improving how we estimate $\widehat{E}^G$ . . . 9
5.2.2.1 Testset method . . . 9
5.2.2.2 n-fold cross-validation . . . 9

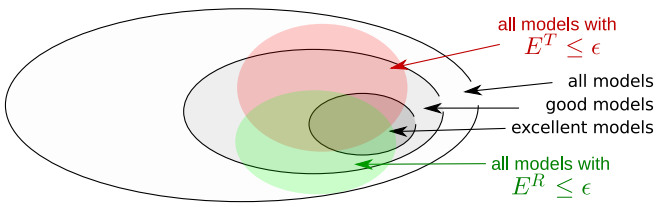
5.3 Nonlinear basis functions **11**
5.3.1 A non-linearly separable problem . . . 11
5.3.2 Nonlinear basis function . . . 11
5.3.3 Preprocessing: Sphering . . . 12
5.3.4 Linear neuron for regression: Analytical solution . . . 13

5.1 Regularization

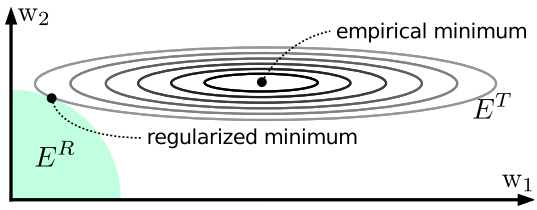
Risk function

$$R_{[\underline{\mathbf{w}}]} = \underbrace{E^T_{[\underline{\mathbf{w}}]}}_{\substack{\text{training}\\ \text{error}\\ \text{e.g. quadratic cost,}\\ \text{cross-entropy,...}}} + \underbrace{\lambda E^R_{[\underline{\mathbf{w}}]}}_{\substack{\text{regularization}\\ \text{term}}} \stackrel{!}{=} \min_{\underline{\mathbf{w}}}$$

- $E^R_{[\underline{\mathbf{w}}]}$: prior knowledge of solution
- λ : regularization parameter



5.1.1 L2 regularization: weight decay



$$E^R_{[\underline{\mathbf{w}}]} = \frac{1}{2}\sum_{(i,j,v',v)} \left(w_{ij}^{v'v}\right)^2 = \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 = \frac{1}{2}\underline{\mathbf{w}}^\top\underline{\mathbf{w}} \tag{5.1.1}$$

Minimization of $R_{[\underline{\mathbf{w}}]}$ through gradient descent

$$\Delta \mathrm{w}_{ij}^{v'v} \sim -\frac{\partial R_{[\underline{\mathbf{w}}]}}{\partial w_{ij}^{v'v}} = \underbrace{-\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_{ij}^{v'v}}}_{\substack{\text{e.g. via}\\ \text{backprop}}} \underbrace{-\frac{\partial E^R_{[\underline{\mathbf{w}}]}}{\partial w_{ij}^{v'v}}}_{\substack{\text{decay}\\ \text{term}}} \tag{5.1.2}$$

L_2 regularization cost:

$$E^R_{[\underline{\mathbf{w}}]} := \frac{1}{2}\sum_{(i,j,v',v)} \left(\mathrm{w}_{ij}^{v'v}\right)^2 = \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 = \frac{1}{2}\underline{\mathbf{w}}^\top\underline{\mathbf{w}} \tag{5.1.3}$$

Computing individual components of the gradient vector:

$$\frac{\partial E^R_{[\underline{\mathbf{w}}]}}{\partial w_{ij}^{v'v}} = \frac{1}{2}\sum_{(k,l,\mu,\gamma)} \frac{\partial}{\partial w_{ij}^{v'v}}\left(w_{kl}^{\mu\gamma}\right)^2 \tag{5.1.4}$$

$$= 0 + \ldots + \frac{1}{2}\frac{\partial}{\partial w_{ij}^{v'v}}\left(w_{ij}^{v'v}\right)^2 + \ldots + 0 = \frac{1}{\cancel{2}}\,\cancel{2}\,w_{ij}^{v'v} \tag{5.1.5}$$

Using vector notation to compute the gradient:

$$\frac{\partial}{\partial \underline{\mathbf{w}}} E^R_{[\underline{\mathbf{w}}]} = \frac{1}{\cancel{2}}\cdot\cancel{2}\cdot\underline{\mathbf{w}} = \underline{\mathbf{w}} \tag{5.1.6}$$

5.1.1.1 Regularized linear neuron for regression: analytical solution

Gradient:

$$\underline{\mathbf{g}}^R := \frac{\partial}{\partial \underline{\mathbf{w}}} R_{[\underline{\mathbf{w}}]} = \frac{\partial}{\partial \underline{\mathbf{w}}} E^T_{[\underline{\mathbf{w}}]} + \lambda \frac{\partial}{\partial \underline{\mathbf{w}}} E^R_{[\underline{\mathbf{w}}]} \tag{5.1.7}$$

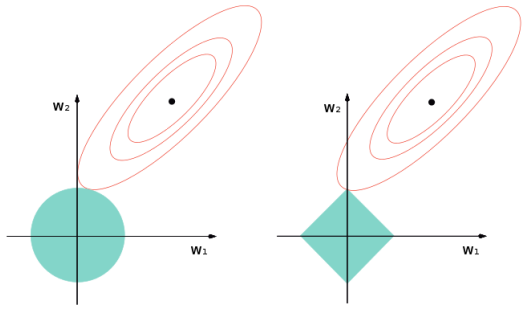
$$= \frac{1}{p}\left(\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top\underline{\mathbf{w}} - \underline{\mathbf{X}}\,\underline{\mathbf{y}}_{\text{True}}^\top\right) + \underline{\mathbf{I}}_N\,\lambda\,\underline{\mathbf{w}} \tag{5.1.8}$$

$$\iff \quad \underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top\underline{\mathbf{w}} - \underline{\mathbf{X}}\,\underline{\mathbf{y}}_{\text{True}}^\top + p\lambda\,\underline{\mathbf{I}}_N\,\underline{\mathbf{w}} \overset{!}{=} \underline{\mathbf{0}} \tag{5.1.9}$$

$$\underline{\mathbf{w}}^* = \underbrace{\left(\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top + p\lambda\,\underline{\mathbf{I}}_N\right)}_{\substack{\text{positive definite and}\\ \text{therefore invertible}}}^{-1}\underline{\mathbf{X}}\,\underline{\mathbf{y}}_{\text{True}}^\top \tag{5.1.10}$$

5.1.2 L1 regularization

$$E^R_{[\underline{\mathbf{w}}]} = \sum_{(i,j,v',v)} \left|w_{ij}^{v'v}\right|^1 \tag{5.1.11}$$



[-2mm] *L_2 weight decay*

L_1 lasso

"Lasso" stands for "**l**east **a**bsolute **s**hrinkage and **s**election **o**perator".

$L2$, weight decay:

- "standard" regularizer
- differentiable (simple integration into gradient descent procedure)
- robust to noise (results in a "distributed" model, redundancies in weights allowed)
- non-sparse

$L1$, Lasso, "sparsify":

- sparse (good for interpretable models, e.g. medical applications)
- non-differentiable at $w_i = 0 \forall i$ (but workaround via subgradient/least angle regression)
- less redundancies between weights $\rightsquigarrow$ more prone to noise

5.2 Validation

Recall: Among the requirements for fitting a model to a desired function $y(\underline{\mathbf{x}}; \underline{\mathbf{w}})$ was having a performance measure, a criterion for *model selection*. Specifically,

the generalization **error** E^G which is defined as:

$$E^G_{[\underline{\mathbf{w}}]} := \langle e \rangle_{y_T, \underline{\mathbf{x}}; \underline{\mathbf{w}}} = \iint d\underline{\mathbf{x}}\, dy_T\; P(y_T, \underline{\mathbf{x}})\, e(y_T, y\,(\underline{\mathbf{x}}; \underline{\mathbf{w}})) \tag{5.2.1}$$

Because $P(y_T, \underline{\mathbf{x}})$ is not known, applying empirical risk minimization (ERM) enables us to approximate $E^G_{[\underline{\mathbf{w}}]}$ by computing the

empirical average $E^T_{[\underline{\mathbf{w}}]}$ using the available training data:

$$\left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)} \right) \right\}, \alpha = 1, \ldots, p.$$

The training error $E^T_{[\underline{\mathbf{w}}]}$ becomes:

$$\text{batch training error:} \quad E^T_{[\underline{\mathbf{w}}]} = \frac{1}{p} \sum_{\alpha=1}^{p} \underbrace{e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}} \right) \right)}_{e^{(\alpha)}} \tag{5.2.2}$$

The above is not limited to scalar attributes y, y_T and extends to $\underline{\mathbf{y}}, \underline{\mathbf{y}}_T \in \mathbb{R}^M$.

where $e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}} \right) \right)$ (or $e^{(\alpha)}$ for brevity) is the cost computed from the prediction for a specific observation $y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})$ and its corresponding label $y_T^{(\alpha)}$. The superscript $^{(\alpha)}$ is used to index a specific point (sample) in the dataset. We have also seen that E^T can be further approximated by changing its "scope" to a set of samples in a mini-batch or a single randomly selected sample (i.e. stochastic online learning)

This leaves us with the question:

Q 5.2.1: *How well does E^T represent E^G?*

We will now focus on:

1. Diagnostics for how well E^T represents E^G
2. Improving estimates for the generalization error E^G
3. Common methods for improving generalization error E^G, doing better on unseen data (e.g. regularization)

Optimizing model parameters to training data and obtaining some solution $\underline{\mathbf{w}}^* = \text{argmin}_{\underline{\mathbf{w}}}\, E^T_{[\underline{\mathbf{w}}]}$ does not imply good generalization performance.

$$\text{small } E^T_{[\underline{\mathbf{w}}^*]} \not\Rightarrow \text{small } E^G_{[\underline{\mathbf{w}}^*]} \tag{5.2.3}$$

Expected:

$$E^T_{[\underline{\mathbf{w}}^*]} < E^G_{[\underline{\mathbf{w}}^*]} \tag{5.2.4}$$

Validation: evaluate the generalization of a model (and its training procedure) by estimating

$$\widehat{E}^G_{[\underline{\mathbf{w}}]} \approx E^G_{[\underline{\mathbf{w}}]} \tag{5.2.5}$$

This implies that we need a set of data.

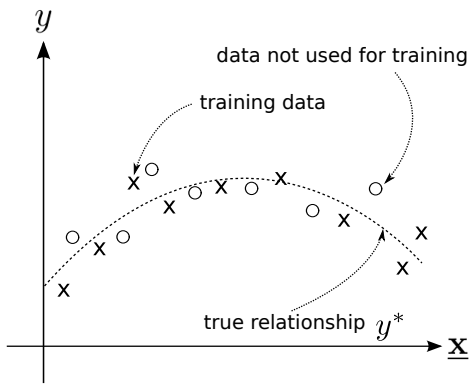
It is very important that we do not use this data for training the model. This data has to be completely excluded from the model selection process.

We use the original training set $\{\underline{\mathbf{x}}^{(\alpha)}_{\text{train}}, y_{T\,\text{train}}^{(\alpha)}\}_{\alpha=1}^{p}$ for model selection. Model selection uses the training data to find $\underline{\mathbf{w}}^* = \text{argmin}_{\underline{\mathbf{w}}}\, E^T_{[\underline{\mathbf{w}}]}$.

We use the new hold-out set to *validate* ($\{\underline{\mathbf{x}}^{(\beta)}_{\text{val.}}, y_{T\,\text{val.}}^{(\beta)}\}_{\beta=1}^{q}$) to estimate the generalization error for this particular solution (i.e. $\widehat{E}^G_{[\underline{\mathbf{w}}^*]}$)

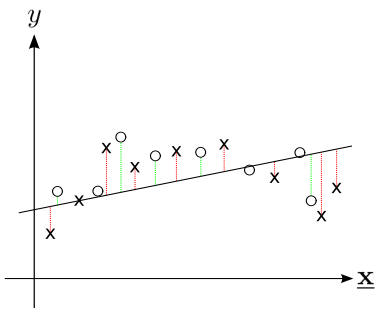
Let us look at the following scenario where we've trained

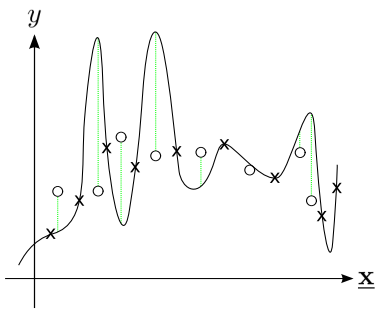
5.2.1 Overfitting vs. Underfitting



data generation:
e.g. $y_T = y^*_{(\underline{\mathbf{x}};\underline{\mathbf{w}})} + \underbrace{\eta}_{\text{noise, zero mean}}$

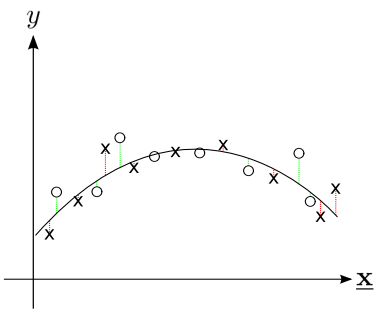
Compare E^T and $\widehat{E}^G$:





$$\left.\begin{array}{l} E^T \text{ large} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \approx \widehat{E}^G \tag{5.2.6}$$

$$\left.\begin{array}{l} E^T \text{ small} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \ll \widehat{E}^G \tag{5.2.7}$$



Diagnostics

$$\left.\begin{array}{l} E^T \text{ small} \\ \widehat{E}^G \text{ small} \end{array}\right\} E^T \approx \widehat{E}^G$$

Q 5.2.2: *What is underfitting?*

- function complexity > model complexity (e.g. fitting a parabola using only a straight line, fitting a sinusoid using a parabola)
- the degrees of freedom in the model are too ...low
- the model is insensitive to variations in the features

Q 5.2.3: *How would replacing a few training samples change the solution?*

- Not much.

Q 5.2.4: *What is overfitting?*

- function complexity < model complexity (e.g. fitting a line using a parameterized sinusoid, using an MLP with non-linear functions to fit a straight line.
- the degrees of freedom in the model are...unnecessarily high
- the model is over-sensitive to the noise component
- the model has "memorized" the training data instead of learning a representation for it.

Q 5.2.5: *How would replacing a few training samples change the solution?*
- A lot.

5.2.1.1 Dealing with overfitting/underfitting

Assuming one has performed diagnostics of comparing E^T and $\widehat{E}^G$ to identify if a model is overfitting or underfitting. How would one mitigate one or the other?

5.2.1.2 Different views of the bias-variance tradeoff

Remember: it's always a bias-variance <u>tradeoff</u>

The above "fixes" rely on having concluded that the model is either overfitting or underfitting. And that this can potentially be mitigated by taking these different measures. However, it is important to understand how overfitting and underfitting reflect where we are on the bias-variance tradeoff. This means

	potentially reduces →	overfitting	underfitting
1	more training data	yes	no
2	larger regularization coeff. λ	yes	no
3	smaller regularization coeff. λ	no	yes
4	use more complex model	no	yes
5	use simpler model	yes	no
6	increase number of hidden neurons in MLP	no	yes
7	add more hidden layers to MLP	no	yes
8	add features to the input $\underline{\mathbf{x}}$	no	yes
9	shuffle the data	no	no
	more techniques?		

that repeatedly applying one solution to an overfitting model can eventually cause the model to underfit and vice-versa. One should also note that if one of the above doesn't work. It does not conclude that the model was not overfitting or underfitting. The above are simply guidelines on how to understand why a model has poor generalization performance and come up with simple techniques for finding a different bias-variance tradeoff.

Although we've just talked about how increasing the size of the training set can help mitigate overfitting. Observing how E^T and $\widehat{E}^G$ behave as a function of the training set size p is a way of determining if an overfitting problem even exists. Keep in mind that $\widehat{E}^G$ would not directly depend on the size of the training set as it is only evaluated on the hold-out set, which we won't change in size. Nevertheless, every time we train our model with a larger training set, we evaluate both E^T and $\widehat{E}^G$ for this trained model.

Change the size of the training set p to identify potential overfitting vs. underfitting of the model:

Q 5.2.6: *What E^T vs. p relationship do we expect for an* underfitting *model?*

Q 5.2.7: *What E^T vs. p relationship do we expect for an* overfitting *model?*

Q 5.2.8: *What $\widehat{E}^G$ vs. p relationship do we expect for an* underfitting *model?*

Q 5.2.9: *What $\widehat{E}^G$ vs. p relationship do we expect for an* overfitting *model?*

Q 5.2.10: *What is the benefit of looking at the bias-variance tradeoff as a function of p?* - Other than disentangling high bias from high variance, we also

get an assessment of how much we would gain in generalization performance

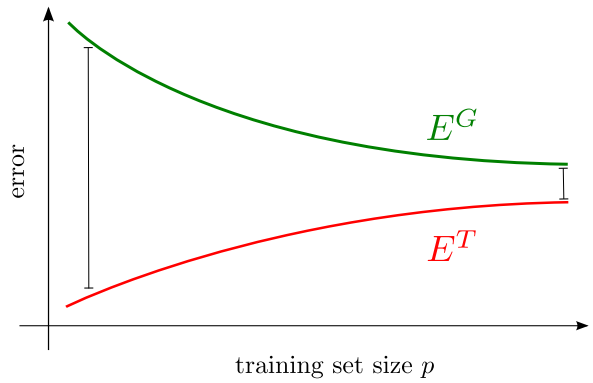


Figure 5.2.1: The bias-variance tradeoff for different number of training samples

if we paid to increase the training set by x% (i.e. cost-benefit analysis).

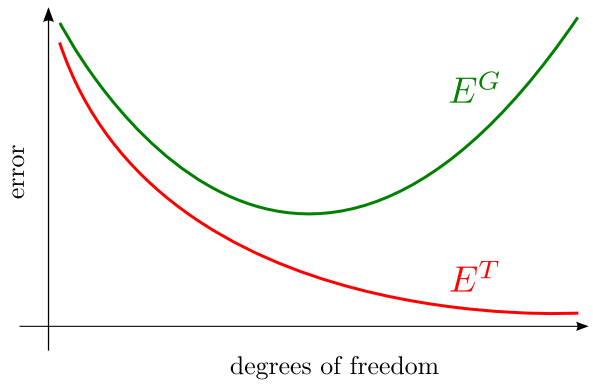


Figure 5.2.2: The bias-variance tradeoff with variable degrees of freedom

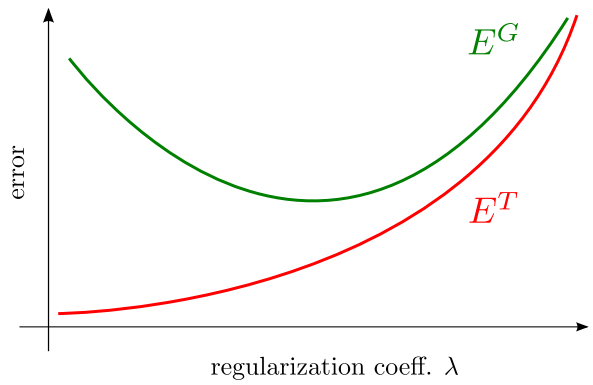


Figure 5.2.3: The bias-variance tradeoff for different regularization strengths

5.2.2 Improving how we estimate $\widehat{E}^G$

Q 5.2.11: *Why should we care about how we estimate $\widehat{E}^G$?* - It's simply to

identify potential overfitting or underfitting. Are we memorizing the training data or did we underestimate the complexity of function we're trying to fit? Once we identify what kind of problem we have with our model it implies which "fixes" are relevant and which would simply not help at all.

5.2.2.1 Testset method

Testset method (with hyperparameters, e.g. regularization coeff. λ)

3-way split of the data.

1. training set: perform model selection for different values of λ
2. test set: select value of λ, which provides best prediction results on this set $\rightsquigarrow \lambda^{\text{opt}}$
3. validation set: estimate generalization performance of selected λ This is data that was not used for model selection or hyperparameter selection.

Potential overfitting of λ^{opt} to test set.

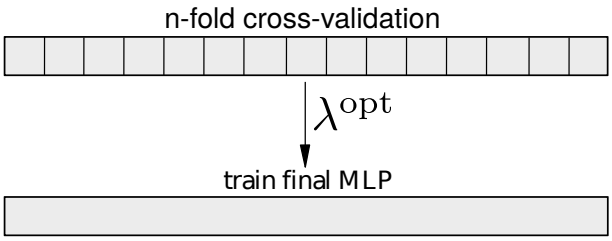
5.2.2.2 n-fold cross-validation

n-fold cross-validation for hyperparameter selection

for $\lambda = \lambda_1$ **to** λ_n **do**
| perform n-fold cross-validation on data with regularization λ
end

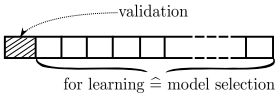
pick optimal λ^{opt} with minimum $\widehat{E}^G = \langle \widehat{E}^G_k \rangle_{n\text{folds}}$

final model: train network on all data with λ^{opt}



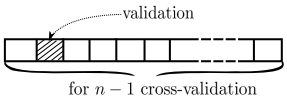
Optional: Additional validation set for estimating $\widehat{E}^G$ of final model.

1



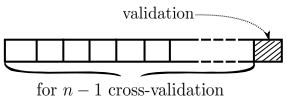
- do n - 1 cross-validation for all values of λ
- train with best λ
- validate with learned model

2



- do n - 1 cross-validation for all values of λ
- train with best λ
- validate with learned model

n



- do n - 1 cross-validation for all values of λ
- train with best λ
- validate with learned model

- $\widehat{E}^G \quad \widehat{=} \quad$ average over all n validation errors

Every cross-validation (inner loop) above yields a model with some generalization performance. The parameters (incl. hyperparameter selection) of each cross-validation can be different from the next. The purpose of the outer loop is to estimate generalization performance that is less sensitive to the selection of the validation set, hyperparameter selection, and model selection procedure.

- Never use test data for model selection.
- Always embed the whole selection procedure (including hyper-parameter search) within an n-fold cross-validation run.
- Watch out for validation data "leaking" into the training set. Basing any decision (e.g. choice of hyperparameter values) based on the validation set is seen as validation data "leaking" into the training data. It has the potential of overfitting to the validation set which defeats the purpose of measuring performance on data that was never seen and did not influence any decisions. Performance that resembles that of the model when it is deployed and used with data outside of our control.

5.3 Nonlinear basis functions

5.3.1 A non-linearly separable problem

Q 5.3.1: *Find a perfect linear separation between the classes using only a connectionist neuron.*

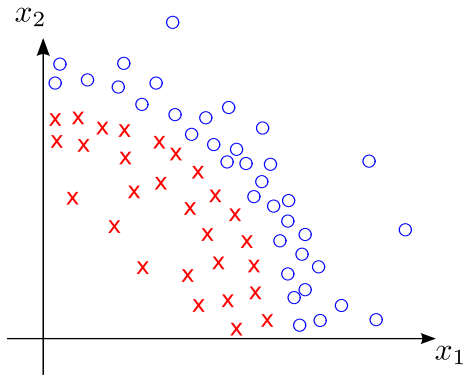


Figure 5.3.1: A non-linearly separable classification problem.

Q 5.3.2: *Fit this parabola using only a connectionist neuron.*

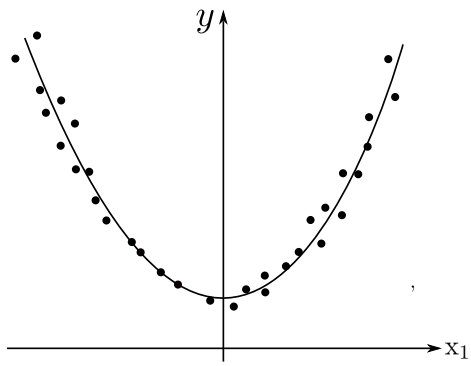


Figure 5.3.2: A non-linearly separable classification problem.

Q 5.3.3: *How would you characterize any parabolic (quadratic) function?*

$$y(x) = a \cdot x^2 + b \cdot x + c = w_2 \cdot x^2 + w_1 \cdot x + w_0 \tag{5.3.1}$$

5.3.2 Nonlinear basis function

Feature transformation ($\underline{\mathbf{x}} \in \mathbb{R}^N$):

$$\underline{\phi} : \underline{\mathbf{x}} \mapsto \underline{\phi}(\underline{\mathbf{x}}) \tag{5.3.2}$$

$\phi(\underline{\mathbf{x}})$ can transform our N-dim input $\underline{\mathbf{x}}$ to another feature space of possibly higher dimensionality.

Expanding our feature space using *Monomials* of highest degree k (i.e. polynomial expansion):

$$\phi_i(\underline{\mathbf{x}}) \in \{\ 1, x_1\ ,\ x_2, \ldots, x_N, x_1^2\ ,\ x_1x_2, \ldots, x_1x_N, \ldots,\ x_N^2,\ \ldots \\ x_1x_N^{k-1}\ ,\ x_1^k, \ldots, x_N^k\}$$

$$\phi_i(\underline{\mathbf{x}}) \in \left\{ \prod_{j=1}^{N} x_j^{a_j} \quad : a_j \in \mathbb{N}_0 \ \text{ with } \ 0 \leq \sum_{j=1}^{N} a_j \leq k \right\} \tag{5.3.3}$$

where $i = 1, \ldots d$.

Example: $k = 9, N = 2 \quad \Rightarrow \quad d = 55$

5.3.3 Preprocessing: Sphering

Large $\|\underline{\mathbf{x}}\|$ can lead to very large $\phi_i(\underline{\mathbf{x}})$.

We want to decorrelate the transformed data and standardize it (keep variance of x_i = 1) *Sphering* is a preprocessing step that accomplishes both. The sphering transformation yields decorrelated input variables each with unit variance. For every point $\alpha = 1, \ldots, p$ from the training set:

$$\underline{\mathbf{x}}^{(\alpha)}_{\text{sphered}} = \underline{\mathbf{\Lambda}}^{-\frac{1}{2}} \underline{\mathbf{E}}^{\top} \underline{\mathbf{x}}^{(\alpha)}_{\text{centered}}. \tag{5.3.4}$$

Here

$\underline{\mathbf{x}}^{(\alpha)}_{\text{centered}} = \underline{\mathbf{x}}^{(\alpha)} - \langle \underline{\mathbf{x}} \rangle$ denotes the centered data point α w.r.t. the center of the training data $\langle \underline{\mathbf{x}} \rangle = \frac{1}{p} \sum_{\alpha=1}^{p} \underline{\mathbf{x}}^{(\alpha)}$,

$\underline{\mathbf{E}} = (\underline{\mathbf{e}}_1, \ldots, \underline{\mathbf{e}}_N)$ is the eigenvector matrix and $\underline{\mathbf{\Lambda}} = \text{diag}(\lambda_1, \ldots, \lambda_N)$ is the eigenvalue matrix for the eigendecomposition

$$\underline{\mathbf{C}}\, \underline{\mathbf{e}}_i = \lambda_i \underline{\mathbf{e}}_i$$

of the covariance matrix $\underline{\mathbf{C}}$ with $C_{ij} = \frac{1}{p} \sum_{\alpha=1}^{p} x^{(\alpha)}_{\text{centered},i}\, x^{(\alpha)}_{\text{centered},j}$.

Examining the properties of the data after sphering:

$$\frac{1}{p} \sum_{\alpha=1}^{p} \underline{\mathbf{x}}^{(\alpha)}_{\text{sphered}} = \underline{\mathbf{0}} \quad \text{(centered)}, \tag{5.3.5}$$

$$\frac{1}{p} \sum_{\alpha=1}^{p} (x^{(\alpha)}_{\text{sphered,i}})^2 = 1 \quad \text{(unit variance)}, \tag{5.3.6}$$

$$\frac{1}{p} \sum_{\alpha=1}^{p} x^{(\alpha)}_{\text{sphered},i}\, x^{(\alpha)}_{\text{sphered},j} = 0 \quad \text{(zero covariance)}, \tag{5.3.7}$$

Important: Reuse the same pre-processing (centering, sphering) on validation set. Do not recompute a separate mean or sphering transformation for the validation set. Also apply to test-set in case of a 3-way split.

5.3.4 Linear neuron for regression: Analytical solution

Data:

transformed input	$\underline{\phi}^{(\alpha)} := \underline{\phi}(\underline{\mathbf{x}}^{(\alpha)}) = \left(\phi_1(\underline{\mathbf{x}}^{(\alpha)}), \phi_2(\underline{\mathbf{x}}^{(\alpha)}), \ldots, \phi_d(\underline{\mathbf{x}}^{(\alpha)})\right)^\top$
label	$y_T^{(\alpha)} \in \mathbb{R}$
with	$\alpha = 1, \ldots, p$ and $\underline{\mathbf{x}}$ has been centered and sphered.

Model:

linear neuron on the *the transformed* input:

$$y(\underline{\phi}; \underline{\mathbf{w}}) = \underline{\mathbf{w}}^\top \underline{\phi} = \sum_{i=1}^{d} w_i \phi_i(\underline{\mathbf{x}}) \quad \text{(bias already incl.)} \tag{5.3.8}$$

quadratic cost function, L_2 regularization. Reuse solution for regularized linear regression:

$$\underline{\mathbf{w}}^* = \left(\underline{\mathbf{\Phi}}\,\underline{\mathbf{\Phi}}^\top + \lambda \underline{\mathbf{I}}_d\right)^{-1} \underline{\mathbf{\Phi}}\,\underline{\mathbf{y}}_{\text{True}}^\top \tag{5.3.9}$$

where $\underline{\mathbf{\Phi}} = \left(\underline{\phi}^{(1)}, \ldots, \underline{\phi}^{(\alpha)}, \ldots, \underline{\phi}^{(p)}\right) \in \mathbb{R}^{d,p}$